We're calling the `continueConversation` function directly from our client component, without needing to set up a separate API endpoint.

The `readStreamableValue` function is designed to work in tandem with `createStreamableValue` on the client side. It's specifically used to read and process the streamable value that was created and updated on the server.

When we use the `readStreamableValue`, we can update the UI incrementally as the AI generates its response, providing a more responsive user experience. Finally, we removed the `useChat` since the whole logic is encapsulated in the server actions directly. This means that we don't have to include this code with the final client, reducing the final bundle size.

With these changes implemented, you can now run the web application locally. You'll notice that while the functionality remains the same from a user's perspective, the interactions with the server actions happen seamlessly in the background. The beauty of this approach is that it maintains the lively, interactive feel of a client-side application while enhancing the performance and security benefits of server-side processing.

The final piece of this puzzle is the ability to stream React components directly from within server actions. This approach is highly recommended, as it uses the full power of RSCs and the Vercel AI SDK. To fully utilize this capability, you'll need to understand some additional concepts, particularly around managing AI and UI state. In the next section, we'll peruse a comprehensive walk-through of managing UI state and explore how to use external tools to enhance this process.

4.1.4 Techniques for generating and streaming UI components

Now that we're using server actions for seamless text streaming, we can unlock perhaps the most influential feature: streaming React components directly in response to AI completions. This capability allows us to generate and update complex UI elements based on AI-generated content in real time.

For example, consider an application that can generate and display a weather forecast. As the large language model (LLM)–powered app processes a user's request for weather information, it could stream a series of React components that progressively build a rich, interactive weather display:

1. Initially, it might render a loading component: `<WeatherLoading />`.
2. Then, as it generates the forecast, it could stream city information: `<CityInfo name="New York" />`.
3. Following that, it might add temperature data: `<TemperatureDisplay high={75} low={62} />`.
4. Finally, it could include a weekly forecast: `<WeeklyForecast data={[...]} />`.

The user would see these components appear and update in real time as the AI generates the information, creating a dynamic and engaging experience.